Computer Science & IT



Data Structure & Programming

Queue

Lecture No. 02



By- Abhishek Sir

Recap of Previous Lecture



- Topic — Queue
- Topic — Linear queue
- Topic — Enqueue Dequeue
- Topic
- Topic

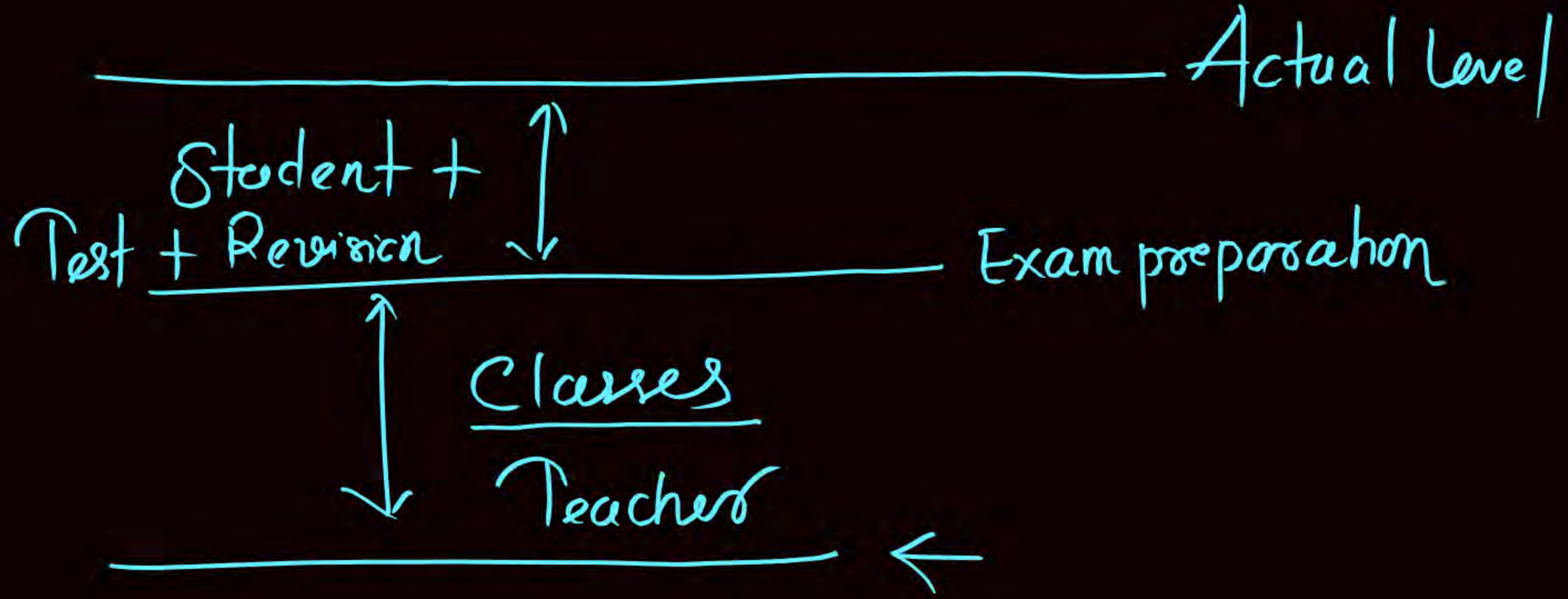
Slide

Topics to be Covered



- Topic — Circular Queue
- Topic — Implementation of queue using stack
- Topic — practice problem
- Topic
- Topic

Slide


Actual Level
Student +
Test + Revision
Exam preparation
Classes
Teacher



Topic : Circular Queue

1	2	3	4	5
0	1	2	3	4

front = 0

rear = 4

· Dequeue ( )

	2	3	4	5
0	1	2	3	4

front = 1

rear = 4

If in this queue if new element Inserted then Linear queue shows error

Slide



Topic : Circular Queue

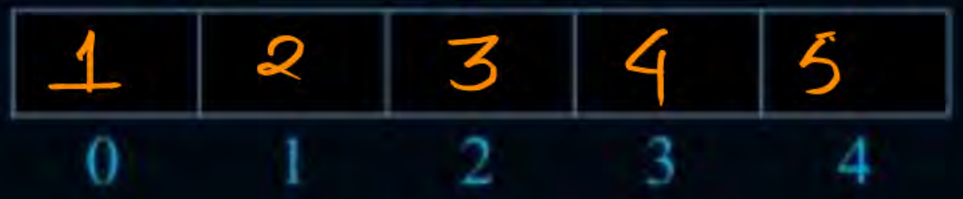
Linear queue flow is the queue declared as full even if Some empty position available at the begining.

Slide



Topic: Circular Queue

front = 0, rear = 4 Enqueue(6)





Queue is full ⇒ (front = 0 && rear = 99)



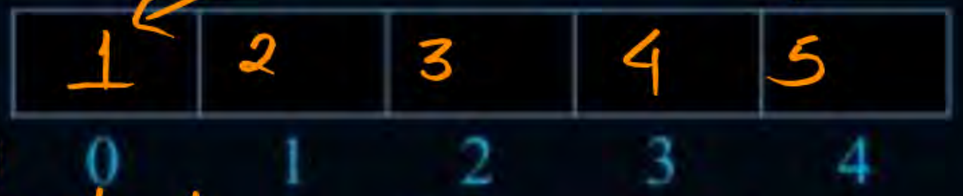


↑ first full condition

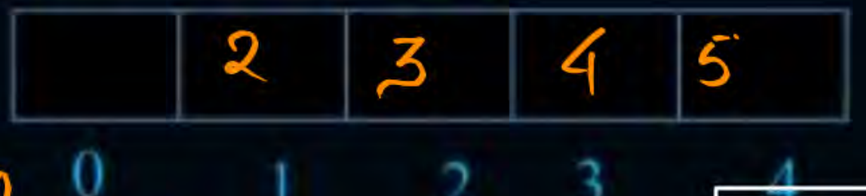
Slide

Topic: Circular Queue

front = 0, rear = 4 Dequeue( )



front = 1, rear = 4



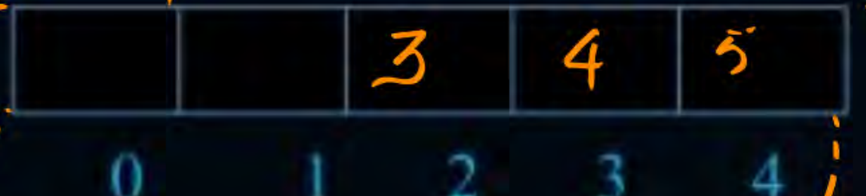
Remainder

rear = (rear + 1) % size

front = 1
rear = 4 Dequeue( )



front = 2
rear = 4



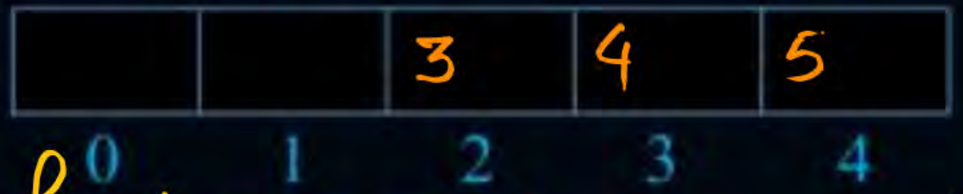
rear = (4+1)%5

5%5 = 0

Slide

Topic: Circular Queue

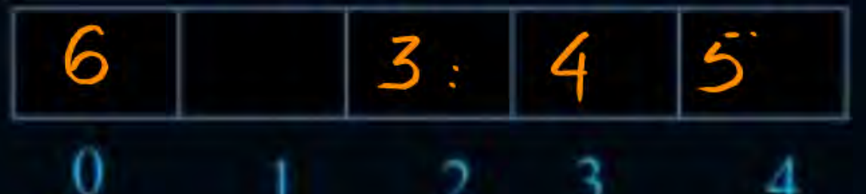
front=2 rear=4 Enqueue(6)



front=2, rear=0 Enqueue(7)



front=2, rear=0



front=2 rear=1



Circular queue

rear Index < front

rear = (rear+1) % Max

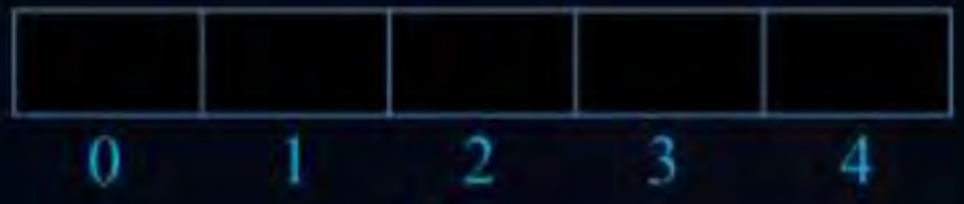
= (0+1) % 5 = 1 % 5 = 1

Slide

Topic: Circular Queue

front = 2, rear = 1 Enqueue(8)





front = 2, rear = 1 ← Queue is full



front = rear + 1

f

r



front = (rear + 1) % Max

(front = 0 && rear = Max - 1) || (front = rear + 1)

Slide



Topic : Circular Queue

```
#define Size 100

void Enqueue(int x){
  if(((front==0 && rear==99) || (front==rear+1))){   // Size - 1
      printf("queue is full");
      return;
  }
  rear=(rear+1)%100; // Size
  a[rear]=x;
  if(front==-1)
      front=0;
}
```

```
front=(rear+1)%Max;
```

Slide



Topic : Circular Queue

```
int dequeue() {
    int data;
  if(front == -1) {
      printf("Empty queue");
      return -1;
  }
    data = a[front];
    if (front == rear)
            front = rear = -1;
    front = (front+1) % 100   // Size
    return data;
}
```

Stack & queue code

Not important from

GATE prespective

Slide

Implementation of Queue Using Stack

Stack — push()
pop()



Stack$_1$

Enqueue(x)
= push(x) ✓

Enqueue(1),
Enqueue(2)
Enqueue(3)
Enqueue(4)

Queue : Enqueue() Single pop()
Dequeue()

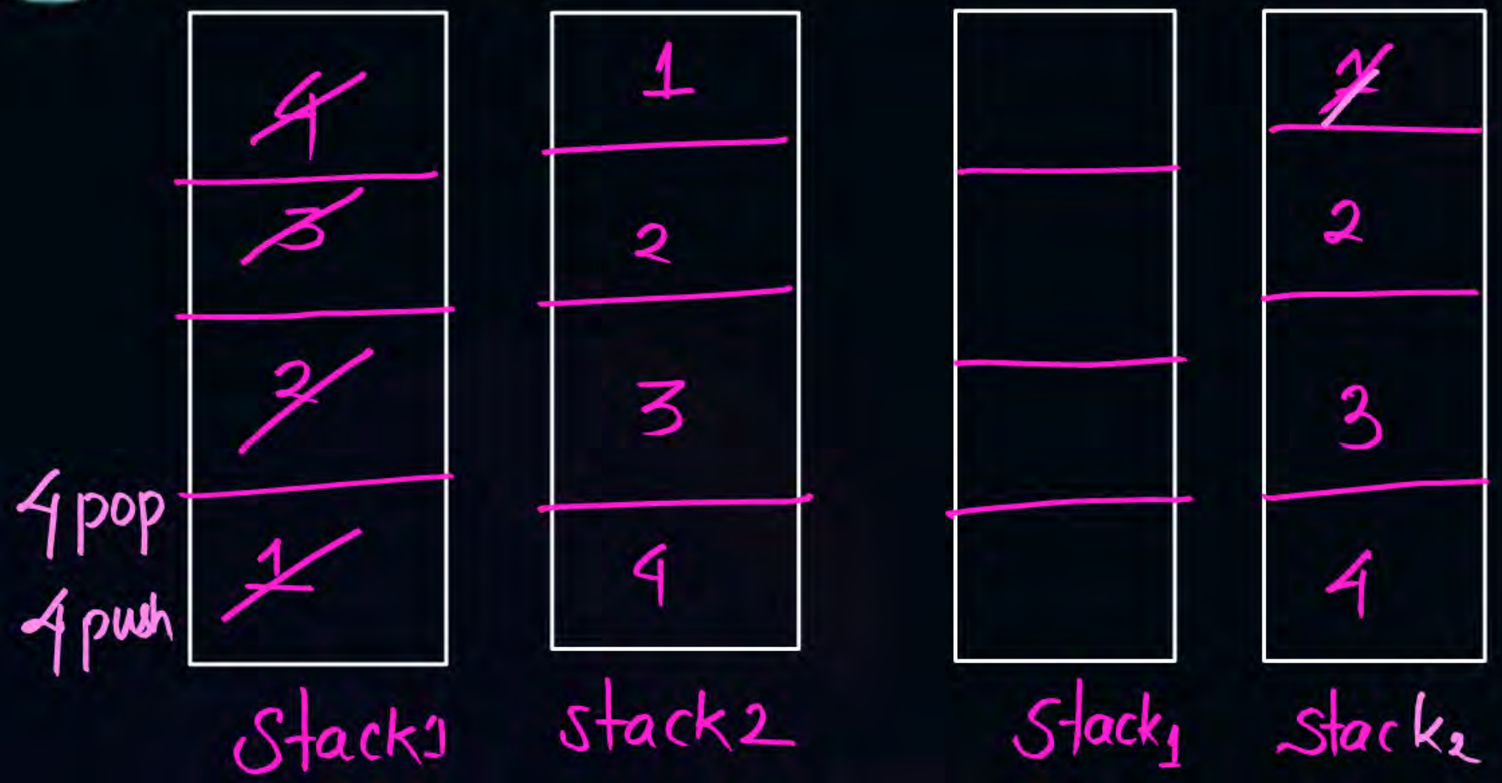
Dequeue is Not equal to pop()

* Take one more stack: Stack$_2$
* Empty containt of Stack$_1$ & put in Stack$_2$ (1) Step
* pop top content of Stack$_2$ (2) step ✓✓
* Restore content of Stack$_1$ (3) Step

Slide

Implementation of Queue Using Stack

Single pop()

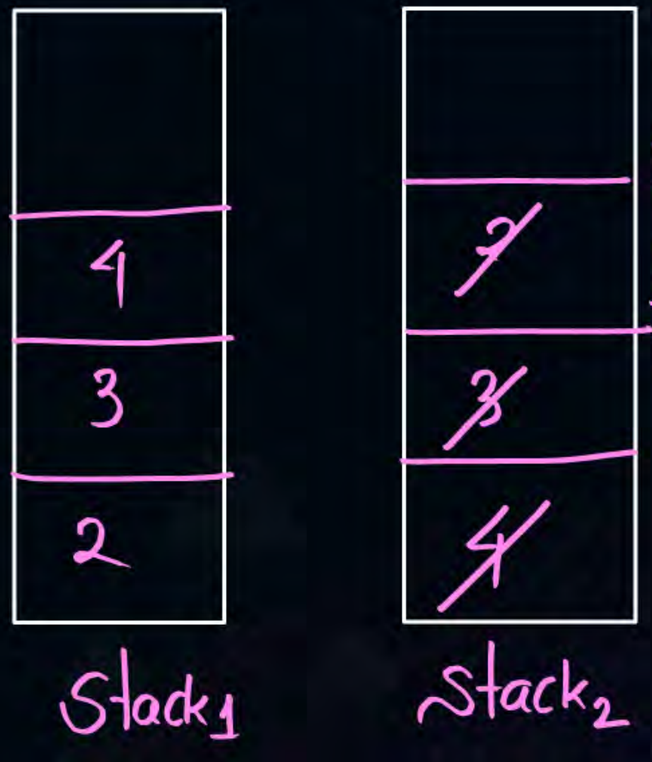


Step 1

npop npush

Step 2

1 pop



Step 3 Restore content of S_1

Slide



Implementation of Queue Using Stack

Question: In above implementation of queue if we perform n - Enqueue operation & 1 dequeue operation then Total No. of push & pop performed is? (considering both stack)

Slide

Implementation of Queue Using Stack

Enqueue() = push(x)

n times Enqueue = n times push

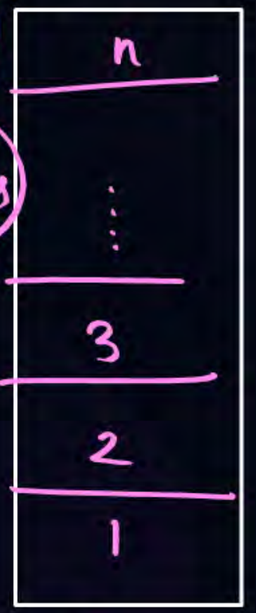
dequeue = push n + n-1 = 2n-1

pop = n + 1 + n-1 = 2n

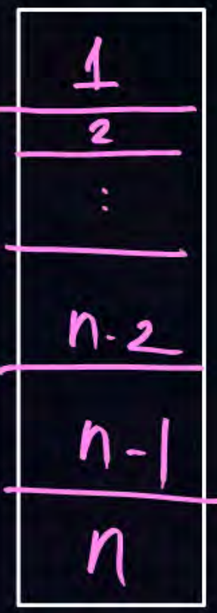
pop = n(S1)

push = (n times)

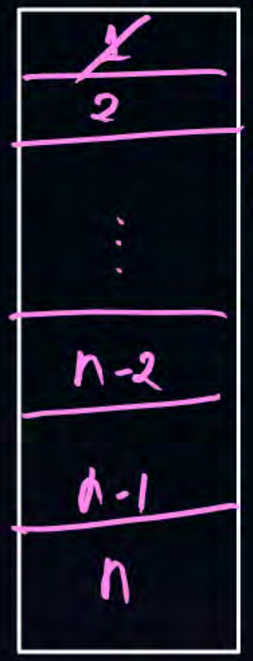
(S2) (1)



S1



S2



S2

1 pop



n-1 pop

S2



n-1 push

S1

Slide



Implementation of Queue Using Stack

n Enqueue
1 Dequeue

total push = n + (2n-1) = 3n-1
(n: Enqueue, 2n-1: Dequeue)

total pop : 2n

n = 100

push 3×100 - 1 = 299

pop = 2×100 = 200

499

n = 99 = 2×99 - 1
= 197

pop = 2n = 2×99
= 198

Slide



Implementation of Queue Using Stack

#Q In above Implementation if we perform 50 Enqueue operation & 2 Dequeue() operation then if total push is x and total pop is y then x+y is ____

NAT

50 Enqueue = # of push 50

1 dequeue – pop = 2n push 2n-1

100 99

2 dequeue: 2×49 = 98 97

149

249

98

97

444

Slide



Implementation of Queue Using Stack

Qushon

above implementation | 100 Enqueue | & 2 dequeue operation performed the total No. of Push & pop operation.

push = 100 Enqueue ✓

1 pop, $2n = 2 \times 100 = 200$, $2n-1$ push = 199

2 pop $2 \times (n-1) = 2 \times 99 = 198$, $2n-1$ push : 197

499
198 ← pop
197 ← pop
894

Slide

Q. Suppose a stack implementation supports an instruction REVERSE, which reverses the order of elements on the stack, in addition to the PUSH and POP instructions. Which one of the following statements is TRUE with respect to this modified stack?

(GATE_2014_2 M)

(A) A queue cannot be implemented using this stack.

(B) A queue can be implemented where ENQUEUE takes a single instruction and DEQUEUE takes a sequence of two instructions.

(C) A queue can be implemented where ENQUEUE takes a sequence of three instructions and DEQUEUE takes a single instruction.

(D) A queue can be implemented where both ENQUEUE and DEQUEUE take a single instruction



2 mins Summary

Topic

Topic

Topic

Topic

Topic

Circular queue

queue Implementation using stack

practice problem

Slide

THANK - YOU

Slide